

COMPTIA SECURITY+

(SY0-701) — COMPLETE STUDY NOTES

Original notes, written in plain language — Harsh Saini / MR CYBER

These notes cover all five Security+ domains in simple, everyday language, with fuller explanations rather than just short definitions — written to actually be understood on a first read, not just memorized. Each domain ends with a quick exam-tip box. Exam format, question count, and passing score change over time, so always confirm the current details on comptia.org before booking your exam.

Domain 1 — General Security Concepts

This domain is the foundation: the core ideas that every other domain builds on. Almost every scenario question later in the exam — an incident, an architecture choice, a compliance issue — ultimately traces back to one of these basic concepts. If you understand this section deeply, the rest of the exam gets noticeably easier.

The CIA Triad

The CIA Triad is the single most repeated idea in all of security. Every control, every policy, and every incident response action exists to protect one (or more) of these three properties. When you're unsure what a security measure is 'for,' ask yourself which piece of CIA it protects — that's usually the fastest way to understand it.

- **Confidentiality** — keeping data secret from people who shouldn't see it. Achieved through encryption, access controls, and permissions. Example: encrypting a laptop's hard drive so a thief who steals it still can't read the files.
- **Integrity** — making sure data hasn't been changed or tampered with, whether by an attacker, a software bug, or a transmission error. Achieved through hashing, digital signatures, and checksums. Example: a downloaded file's hash not matching the publisher's listed hash tells you the file was altered somewhere along the way.
- **Availability** — making sure systems and data are accessible to the people who legitimately need them, when they need them. Achieved through redundancy, backups, and DDoS protection. Example: a hospital's patient record system needing 24/7 uptime, since downtime can directly delay care.
- **Non-repudiation** — a bonus concept often paired with CIA — proof that someone really did perform an action, so they can't credibly deny it later. Digital signatures are the classic example: if a document is signed with someone's private key, they can't later claim they didn't sign it, because only their key could have produced that signature.

AAA Framework

AAA describes the three-step process behind almost every login and access decision you'll ever encounter, from unlocking a phone to accessing a corporate database.

- **Authentication** — proving you are who you say you are (password, fingerprint, token). This only confirms identity — it says nothing yet about what you're allowed to do.
- **Authorization** — determining what an authenticated user is actually allowed to do. Being authenticated (logged in) doesn't automatically mean you're authorized to access everything — a regular employee might log in successfully but still be blocked from HR's salary database.
- **Accounting** — logging what a user did, when, so there's a record to review later. This is what makes incident investigations possible after the fact — without accounting, you'd have no way to reconstruct what actually happened.

Simple way to remember it: Authentication is 'who are you,' Authorization is 'what can you do,' Accounting is 'what did you do.'

Security Controls

Controls can be grouped two different ways, and the exam expects you to know both groupings and be able to classify a real-world example into the correct bucket for each. Category answers 'what kind of control is this' (technical, managerial, operational, physical). Type answers 'what does this control actually try to do' (prevent, detect, correct, deter, compensate). A single control, like a firewall, is both a category (technical) and a type (usually preventive) at the same time — these two classifications aren't mutually exclusive, they're two separate lenses on the same control.

- **Technical controls** — enforced by systems — firewalls, encryption, antivirus. These run automatically without needing a person to actively decide anything in the moment.
- **Managerial controls** — policies and procedures created by leadership — a password policy, a risk assessment process. These set the rules that technical and operational controls then carry out.
- **Operational controls** — day-to-day practices carried out by people — security awareness training, guard patrols. The key difference from technical controls is that a human actively does the work.
- **Physical controls** — controls you can physically touch — locks, fences, badge readers. These protect the physical environment itself, not just data or systems.

By purpose (what the control is trying to accomplish):

- **Preventive** — stops an incident before it happens (a firewall rule blocking traffic). The goal is to never let the bad thing occur at all.
- **Detective** — identifies an incident that's already happening or already happened (an IDS alert). It doesn't stop anything by itself — it just makes sure someone finds out.
- **Corrective** — fixes things after an incident (restoring from backup). This is about recovery, not prevention.
- **Deterrent** — discourages an attacker without physically stopping them (a warning sign, visible cameras). It works purely on psychology — making the attack look less appealing or riskier than it's worth.
- **Compensating** — an alternative control used when the ideal control isn't possible (extra monitoring on a system that can't be patched yet). This is a 'best available substitute,' used when the textbook-correct fix isn't feasible right now.

Zero Trust

Zero Trust is the modern security model that has largely replaced the older 'castle and moat' approach, where anything inside the network perimeter was automatically trusted. Under Zero Trust, no device or user is trusted by default, even if they're already inside the network — every single request gets verified on its own merits. Authentication, authorization, and device health (is this device patched? does it have security software running?) are all checked continuously, not just once at initial login. The guiding phrase is 'never trust, always verify.' This model exists because so many real breaches happen after an attacker gets just one foothold inside a network — under the old model, that single foothold often meant free movement everywhere; under Zero Trust, the attacker still has to fight for every additional step.

Physical Security Basics

Physical security is easy to overlook when studying because it feels less 'technical,' but a huge number of real breaches start with physical access — a stolen laptop, an unlocked server room, a tailgating attacker walking in behind an employee.

- Badge readers and locks control who can physically enter a space, forming the first layer of defense before any digital control even comes into play.
- A mantrap is a small space with two interlocking doors — only one opens at a time, so a person has to be individually verified before the second door unlocks, specifically preventing tailgating.
- Cameras (CCTV) act as both a deterrent (their visible presence discourages bad behavior) and a detective control (footage lets you investigate after the fact).
- Bollards and fencing prevent vehicle-based physical attacks on a building, such as someone deliberately driving a vehicle into an entrance.

Cryptography Fundamentals

Cryptography underpins almost everything else in this domain — encryption, secure protocols, certificates, and digital signatures are really just different applications of a small set of core cryptographic ideas. Getting comfortable with these basics makes many later topics click into place much faster.

- **Symmetric encryption** — the same single key both encrypts and decrypts data. It's fast and efficient, but creates a real problem: that one key must somehow be shared securely between both parties before they can communicate, and if it's intercepted in transit, the whole scheme is broken (example: AES).
- **Asymmetric encryption** — uses a mathematically linked key pair — a public key (which can be shared openly with anyone) and a private key (which must be kept completely secret). Data encrypted with the public key can only be decrypted with the matching private key. It's slower than symmetric encryption, but it elegantly solves the key-distribution problem, since the public key never needs to be kept secret (example: RSA).
- **Hashing** — turns data of any size into a fixed-length 'fingerprint.' It's one-way — you cannot reverse a hash back into the original data, which is very different from encryption (which is meant to be reversible with the right key). Hashing is used to verify integrity: if you hash a file today and hash it again next week and get a different result, you know the file changed (example: SHA-256).
- **Digital signatures** — combine hashing and asymmetric encryption together. The sender hashes their message, then encrypts that hash with their own private key. Anyone with the sender's public key can verify the signature — proving both that the message wasn't altered (integrity) and that it genuinely came from that sender (authentication/non-repudiation).
- **PKI (Public Key Infrastructure)** — the entire surrounding system of certificates, certificate authorities (CAs), and key management that makes asymmetric encryption practically trustworthy at internet scale. Without PKI, you'd have no reliable way to know that a given public key actually belongs to the person or website it claims to — PKI is what makes HTTPS trustworthy in your browser.
- A certificate authority (CA) is a trusted third party that verifies identities and then issues and digitally signs certificates, essentially vouching 'we checked, and this public key really does belong to this website/organization.'
- A digital certificate binds a public key to a specific identity (like a website's domain name) and is itself signed by a CA — your browser trusts the certificate because it trusts the CA that signed it.

Change Management

Change management is a formal process for making changes to systems safely, rather than letting anyone modify production systems on a whim. A typical process includes: a formal request describing the proposed change, a review/approval step by the right stakeholders, testing in a non-production environment first, careful implementation with a rollback plan ready, and documentation of exactly what changed. Skipping this process is one of the most common real-world root causes of security incidents — an unreviewed, untested change can accidentally open a vulnerability, break a security control, or cause an outage, all without any attacker involved at all.

EXAM TIP

CIA Triad and AAA are asked about constantly, often as scenario questions — know which term applies to a described situation, not just the bare definitions.

Control category (technical/managerial/operational/physical) vs control type (preventive/detective/corrective/deterrent/compensating) is a very common 'pick the best answer' trap — remember a control can be classified both ways at once.

Symmetric = one key, fast, key-sharing problem. Asymmetric = two keys, slower, solves key distribution. Hashing = one-way, checks integrity, is NOT encryption (it can't be reversed).

Domain 2 — Threats, Vulnerabilities & Mitigations

This domain is about knowing your enemy: who attacks, how they attack, what weaknesses they're exploiting, and what you can actually do about it. Expect a lot of scenario-based questions here — 'given this description, which attack or

actor is being described' is one of the most common formats on the real exam, so recognizing patterns matters more than memorizing definitions word for word.

Threat Actors

Different attackers have very different motivations, resources, and skill levels — and understanding an attacker's profile helps predict what they're likely to target and how sophisticated their techniques will be.

- **Nation-state** — government-backed, highly resourced, and patient, often pursuing espionage, intellectual property theft, or infrastructure disruption over long timeframes. Advanced Persistent Threats (APTs) — attackers who maintain long-term, low-profile access — usually fall into this category.
- **Organized crime** — financially motivated groups running security threats like a business — ransomware gangs, fraud rings, and stolen-data marketplaces are typical examples. Their goal is almost always profit.
- **Hacktivist** — motivated by a political or social cause rather than money — website defacement or leaking documents to make a statement are common hacktivist tactics.
- **Insider threat** — someone who already has legitimate access and misuses it. This can be malicious (a disgruntled employee deliberately stealing data) or simply careless (an employee who falls for phishing or misconfigures a system) — both are still classified as insider threats because the risk originates from inside the organization's own trust boundary.
- **Script kiddie** — someone with low technical skill who uses existing, pre-built tools without deeply understanding how they work, often motivated by curiosity or bragging rights rather than serious financial or political goals.
- **Shadow IT** — not really an 'attacker' in the traditional sense, but a real risk category — employees adopting unapproved tools or cloud services without IT's knowledge, creating systems that fall completely outside normal monitoring and security controls.

Social Engineering

Social engineering attacks exploit human psychology — trust, urgency, fear, curiosity — rather than a technical flaw in a system. This is precisely why social engineering remains effective even against organizations with excellent technical defenses: you can patch a server, but you can't 'patch' human nature the same way, which is why awareness training matters so much.

- **Phishing** — a fraudulent email trying to trick someone into clicking a malicious link, downloading an attachment, or giving up credentials, usually by impersonating a trustworthy source.
- **Spear phishing** — phishing that's specifically targeted at a particular person, using personal or organizational details to make the message far more believable than a generic phishing blast.
- **Whaling** — spear phishing specifically aimed at executives or other high-value targets, since compromising a CEO or CFO's account or trust often has much bigger payoff for the attacker.
- **Vishing / Smishing** — the same manipulation tactics as phishing, but carried out over voice calls (vishing) or SMS text messages (smishing) instead of email.
- **Pretexting** — inventing a believable false scenario or identity to manipulate someone into giving up information they otherwise wouldn't — for example, impersonating IT support and 'needing' someone's password to fix an issue.
- **Tailgating / Piggybacking** — following an authorized person through a secure door without using your own access credentials. Piggybacking is when the authorized person knowingly lets you in (holding the door), while tailgating is sneaking in without their awareness at all.
- **Business Email Compromise (BEC)** — impersonating a trusted contact — often via a lookalike or spoofed email address — to trick an employee into an unauthorized action, most commonly a fraudulent wire transfer.

Malware Types

Malware is a broad umbrella term for any malicious software, but the exam expects you to distinguish between specific categories based on how each one spreads and what it's designed to do — these behavioral differences are usually the actual test question, not just the plain definition.

- **Virus** — attaches itself to a legitimate file and requires that host file to actually be run by a user in order to activate and spread — a virus can't spread entirely on its own.
- **Worm** — spreads on its own across a network without needing a host file or any user action at all, which is what makes worms capable of spreading extremely fast.
- **Trojan** — disguises itself as legitimate, desirable software to trick a user into voluntarily installing it — the malicious payload is hidden inside something that looks harmless or even useful.
- **Ransomware** — encrypts a victim's files and then demands payment (usually in cryptocurrency) in exchange for the decryption key — one of the most financially damaging malware categories in recent years.
- **Spyware** — secretly monitors and collects a user's activity or data, often without any visible symptoms, making it hard to detect.
- **Rootkit** — hides itself deep within a system, often at the operating-system or kernel level, specifically to maintain long-term, hard-to-detect access — rootkits are designed around evading normal detection tools.
- **Keylogger** — records everything a user types on their keyboard, most often used to silently steal usernames, passwords, and other sensitive typed information.
- **Botnet** — a network of many compromised devices controlled remotely by an attacker, typically used to carry out large-scale coordinated actions like DDoS attacks or mass spam campaigns.

Application Attacks

These attacks target weaknesses in how software applications are built, most commonly web applications that don't properly validate or sanitize user input.

- **SQL Injection** — inserting malicious SQL code through a form field or input that the application doesn't properly sanitize, letting an attacker manipulate the underlying database directly — potentially reading, modifying, or deleting data far beyond what the application was ever meant to expose.
- **Cross-Site Scripting (XSS)** — injecting malicious script into a webpage so that it runs inside another user's browser when they view that page, potentially stealing their session or performing actions on their behalf.
- **Buffer Overflow** — sending more data than a program's memory buffer is designed to hold, which can overwrite adjacent memory — in the worst case, letting an attacker get their own code executed by the system.
- **Race Condition** — exploiting the small timing gap between when a system checks a condition and when it actually acts on that condition, letting an attacker slip in an unauthorized action during that narrow window.
- **Directory Traversal** — manipulating a file path (often using sequences like '../') to escape the intended folder and access files elsewhere on the system that should have been off-limits.

Network-Based Attacks

Network-level attacks are covered in real depth in the separate Networking notes on the site — DoS/DDoS, man-in-the-middle, ARP/DNS spoofing, and wireless attacks all belong here. For Security+ specifically, the exam expects you to connect each of these network-layer concepts to its actual security impact (confidentiality loss, availability loss, etc.) rather than just recognizing the attack name in isolation.

Vulnerabilities

A vulnerability is a weakness that could potentially be exploited — it's important to understand that a vulnerability by itself isn't necessarily a crisis; it only becomes a real problem when a credible threat actor has both the motive and the means to exploit it.

- **Zero-day** — a vulnerability that's unknown to the software vendor and has no official patch available yet — this is the most dangerous category precisely because there is no ready-made fix, leaving defenders reliant on other mitigations (like network segmentation or monitoring) until a patch is released.
- **Misconfiguration** — a system that's set up incorrectly — default passwords left unchanged, unnecessary ports left open, overly permissive access settings. This is one of the most common real-world causes of actual breaches, often more common than sophisticated zero-day exploits.
- **Unpatched software** — a known vulnerability where a fix already exists from the vendor, but that fix simply hasn't been applied yet to the affected system — the difference from zero-day is important: here, a solution is available, it just hasn't been used.
- **Supply chain vulnerability** — a weakness introduced indirectly through a third-party vendor, software library, or piece of hardware that an organization depends on — meaning a company can be compromised through no direct fault of its own internal systems at all.

Mitigation Techniques

No single mitigation fully eliminates risk on its own — real-world security relies on layering several of these together, an idea often called 'defense in depth.'

- Patch management — keeping software updated on a regular, tested schedule, closing known vulnerabilities before they can be exploited.
- Segmentation — splitting a network into smaller, isolated zones so that a breach in one area doesn't automatically spread everywhere else.
- Least privilege — giving users and systems only the access they actually need to do their job, nothing more, which limits how much damage any single compromised account can cause.
- Monitoring and logging — ensuring unusual activity actually gets noticed by someone, rather than going unseen for weeks or months (the average time-to-detect a breach in the real world is often shockingly long).
- Security awareness training — since so many attacks, phishing especially, target people directly rather than technical systems, an educated workforce is itself a meaningful control.

EXAM TIP

The exam loves 'read a scenario, name the attack' questions — practice recognizing malware and social engineering types from a short description, not just reciting the definition.

Know the difference between a vulnerability (a weakness), a threat (something/someone that could exploit that weakness), and risk (the combined likelihood and impact if it actually does).

Zero-day = no patch exists yet anywhere. This is different from simply 'unpatched,' where a fix does exist but hasn't been applied to that particular system.

Domain 3 — Security Architecture

This domain covers how systems and networks are actually designed and protected at a structural level — less about individual attacks, more about how you build an environment that's resilient and secure by design from the start.

Cloud & Deployment Models

Understanding where infrastructure lives, and who's responsible for securing which layer of it, is core to modern security work — most organizations today run at least some workloads in the cloud.

- **On-premises** — infrastructure owned, housed, and run entirely by your own organization, giving full control but also full responsibility for every layer of security.
- **Cloud** — infrastructure owned and operated by a third-party provider (AWS, Azure, GCP) that you rent access to, rather than owning the physical hardware yourself.

- **Hybrid** — a mix of on-premises and cloud infrastructure, often used during a gradual migration, or to keep especially sensitive systems on-prem for compliance reasons while less sensitive workloads run in the cloud.
- **IaaS (Infrastructure as a Service)** — you rent raw computing resources — virtual servers, storage, networking — and are responsible for managing the operating system and everything installed above it yourself.
- **PaaS (Platform as a Service)** — the provider manages the underlying operating system and runtime environment for you; you're only responsible for deploying and maintaining your own application code on top of it.
- **SaaS (Software as a Service)** — you simply use a finished, ready-made application (like Gmail or Salesforce) — the provider manages everything underneath, including the application itself; your responsibility is mostly limited to how you configure and use it.

Shared responsibility model: in any cloud arrangement, the provider secures the underlying physical infrastructure, but you remain responsible for securing your own data, access configuration, and how you use the service — 'the cloud is secure' does not mean 'whatever I put in the cloud is automatically secure.'

Security Infrastructure Components

Firewalls, IDS/IPS, and proxies are explained in real depth in the Networking notes on the site — this domain builds on that by expecting you to know where in an architecture each component should actually be placed, not just what it technically does in isolation.

- **Secure protocols** — consistently using the encrypted version of a protocol wherever one exists — HTTPS instead of plain HTTP, SFTP instead of plain FTP, SSH instead of Telnet. This single habit closes an enormous number of otherwise-easy interception opportunities.
- **Network Access Control (NAC)** — checks a device's security posture — its patch level, whether antivirus is active and updated — before allowing it onto the network at all, preventing an unhealthy or non-compliant device from ever connecting in the first place.

Data Protection

Not all data carries the same level of risk if exposed, which is why classifying and protecting data appropriately — rather than treating everything identically — is a core architectural concern.

- **Data classification** — labeling data by sensitivity level (public, internal, confidential, restricted) so that it automatically gets handled with an appropriate level of protection, rather than everyone guessing case by case.
- **PII (Personally Identifiable Information)** — any data that can identify a specific individual — full name, Social Security number, home address — and therefore needs strong protection under most privacy regulations.
- **PHI (Protected Health Information)** — health-related data tied to a specific individual, which carries its own dedicated legal protections (such as HIPAA in the United States) on top of general PII protections.
- **Encryption at rest** — encrypting data while it's sitting in storage — on a disk, in a database — so that if the storage medium itself is stolen or improperly accessed, the data is still unreadable.
- **Encryption in transit** — encrypting data while it's actively moving across a network — this is exactly what TLS/HTTPS does for web traffic, protecting data from interception mid-journey.
- **Data Loss Prevention (DLP)** — tools and policies that actively detect and block sensitive data from leaving an organization improperly — for example, automatically blocking an outgoing email that contains what looks like a credit card number.

Resilience & Recovery

This section is about planning for failure ahead of time rather than being caught off guard by it — hardware breaks, ransomware happens, natural disasters occur, and a mature security architecture assumes these things will eventually happen rather than hoping they won't.

- **Redundancy** — having backup components ready — extra servers, extra power supplies, extra network paths — so that a single point of failure doesn't cause a full outage.

- **High availability (HA)** — designing a system specifically to stay operational with minimal downtime, typically achieved through redundancy combined with automatic failover to a backup component the moment the primary one fails.
- **Backups** — separately stored copies of data that let you restore information if the original is lost or corrupted. Full backups copy everything each time; incremental backups only copy what's changed since the last backup (fast, but restoring requires the full chain); differential backups copy everything changed since the last full backup (a middle ground between the two).
- **Disaster Recovery (DR)** — the specific plan for restoring IT systems and data after a major disruptive event — fire, ransomware, flooding, or another disaster.
- **Business Continuity Plan (BCP)** — the broader organizational plan for keeping the entire business functioning during a disruption, not just the IT systems — DR is essentially a subset that lives inside the larger BCP.
- **RTO (Recovery Time Objective)** — how quickly a system needs to be restored and operational again after an incident, expressed as a time duration (e.g. 'must be back up within 4 hours').
- **RPO (Recovery Point Objective)** — how much data loss, measured in time, is acceptable if a system needs to be restored (e.g. 'we can afford to lose at most 1 hour of data') — this figure directly determines how frequently backups actually need to run.

EXAM TIP

Know the cloud service models cold — IaaS/PaaS/SaaS and exactly 'who manages what' at each layer comes up constantly, often as a table-matching or scenario question.

RTO is about how quickly you need to recover (time-to-restore); RPO is about how much data you can afford to lose (time-to-last-backup). Don't mix these two up.

Encryption at rest vs in transit is an easy point if you just remember: at rest = stored somewhere, in transit = actively moving across a network.

Domain 4 — Security Operations

This is the hands-on, day-to-day domain — hardening systems, watching for problems, and responding when something actually goes wrong. Of all five domains, this one is closest to what a working SOC analyst or security operations team does on an average day.

Hardening

Hardening is the general practice of reducing a system's attack surface — the fewer unnecessary things a system exposes, the fewer opportunities an attacker has to find a way in.

- Removing or disabling unnecessary services, ports, and default accounts reduces the number of ways an attacker could potentially interact with a system.
- Applying secure baseline configurations consistently across similar systems, so that security settings don't quietly drift or vary system to system over time.
- Regular patch management, ideally tested in a non-production environment before wide deployment, so that a patch doesn't accidentally break something critical.
- Changing default credentials on any new device or system before it goes into production — default logins are extremely well known and are one of the first things automated scanning tools check for.

Identity & Access Management (IAM)

IAM covers everything involved in confirming who a user is and controlling exactly what they can access — arguably one of the highest-impact areas in all of security, since so many real breaches trace back to a compromised or overly-privileged account.

- **Multi-Factor Authentication (MFA)** — requiring two or more different proof types to log in, typically drawn from these categories: something you know (a password), something you have (a phone or hardware token), and something you are (a fingerprint or other biometric). Even if one factor (like a password) is stolen, MFA still blocks the attacker without the second factor.
- **Single Sign-On (SSO)** — logging in once to gain access to multiple connected systems and applications, instead of maintaining separate logins for each one — improves both convenience and, if implemented well, security (fewer passwords for users to reuse or write down).
- **Federation** — extending authentication trust across separate organizations, letting someone log into a partner organization's system using their own home organization's credentials, without that partner ever needing to store or manage the password directly.
- **Privileged Access Management (PAM)** — extra controls, monitoring, and often time-limits placed specifically around high-privilege accounts like administrators, since these accounts cause the most damage if compromised.
- **Least privilege & role-based access** — users are only granted the access tied to their specific job role, by default nothing extra — reducing both the chance of accidental misuse and the potential damage from a compromised account.

Vulnerability Management

This is an ongoing cycle, not a one-time event — new vulnerabilities are discovered constantly, so vulnerability management has to be a continuous process rather than something done once and forgotten.

- Vulnerability scanning uses automated tools to check systems against a known database of vulnerabilities, quickly surfacing likely weaknesses across an entire environment.
- Penetration testing goes further than scanning — a tester actually attempts to exploit discovered vulnerabilities to prove real-world impact, rather than just flagging a theoretical weakness.
- Findings get prioritized, often using a standardized scoring system like CVSS (Common Vulnerability Scoring System), and then tracked through to remediation rather than just reported and forgotten.

Monitoring & Logging

You can't detect what you don't watch — monitoring and logging are what turn an invisible compromise into something a defender can actually notice and respond to.

- Centralized logging and SIEM tools (covered in depth in the Networking notes) are how most real-world detection actually happens in practice, by correlating events across many different systems at once.
- Establishing a baseline of 'normal' behavior first is essential — anomalies are only meaningful once you actually know what typical, everyday activity looks like for that specific environment.
- Alert fatigue is a real and common problem — when there are too many low-value or false-positive alerts, analysts can become desensitized and genuinely miss a real incident buried in the noise.

Incident Response Lifecycle

This six-step lifecycle is one of the most consistently tested sequences on the entire exam — questions frequently describe a scenario partway through an incident and ask what the correct next step should be, so knowing the order well (not just the individual steps) really matters.

- **1. Preparation** — having a documented plan, the right tools, and trained people ready before anything actually happens — this step happens continuously, well before any specific incident occurs.
- **2. Identification** — detecting and confirming that a security incident is actually occurring, distinguishing a real incident from a false alarm.
- **3. Containment** — limiting the damage — isolating affected systems (for example, disconnecting them from the network) so the problem can't spread any further while it's being dealt with.

- **4. Eradication** — removing the actual root cause from the environment entirely — deleting malware, closing the exploited vulnerability, removing unauthorized access.
- **5. Recovery** — carefully restoring affected systems back to normal, verified-safe operation, rather than just turning them back on and hoping for the best.
- **6. Lessons learned** — reviewing what happened afterward and using it to improve the process going forward — this is often the most frequently skipped step in the real world, despite being one of the most valuable.

Digital Forensics Basics

Forensics is about investigating an incident in a way that preserves evidence properly, which matters both for accurately understanding what happened and for potentially supporting legal action afterward.

- **Chain of custody** — a documented, unbroken record of exactly who handled a piece of evidence, when, and how — this is critical if the evidence ever needs to hold up in a legal proceeding, since any gap in the chain can call the evidence's integrity into question.
- **Order of volatility** — the principle of collecting the most fragile, quickly-disappearing evidence first. The typical order is: CPU registers and cache, then RAM, then active network connections and running processes, and only afterward the data on disk, which is comparatively stable and can safely wait.
- Never work directly on the original evidence — always work from a verified forensic copy or image, preserving the original untouched in case it's needed again later.

Automation in Operations

SOAR (Security Orchestration, Automation, and Response) tools automate repetitive, well-defined response steps — for example, automatically isolating a device the moment it trips a certain high-confidence alert. This frees up human analysts to spend their limited time and attention on the judgment calls that genuinely require a person, instead of manually repeating the same routine steps for every single alert.

EXAM TIP

Memorize the incident response order — Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned. Questions often describe a partial scenario and ask what the correct next step is.

Order of volatility is a classic forensics question — the more likely data is to disappear quickly (RAM before disk), the earlier in the process you need to collect it.

Know the difference between a vulnerability scan (automated, finds likely weaknesses) and a penetration test (manual, actually exploits them to prove real-world impact).

Domain 5 — Security Program Management & Oversight

The least purely 'technical' domain on the exam, but still tested heavily — this domain is about how security actually gets governed, measured, funded, and justified at an organizational level, which matters just as much as the technical controls themselves in a real security program.

Governance Documents

These four document types form a clear hierarchy, from broad intent down to specific instructions, and the exam frequently tests whether you can correctly sort a given example into the right level.

- **Policy** — a high-level statement of intent and rules, setting the 'what and why' ('all laptops must be encrypted') without necessarily specifying exactly how.
- **Standard** — specific, mandatory requirements that support a policy, filling in the exact detail ('all laptops must use AES-256 encryption specifically').
- **Procedure** — step-by-step instructions for actually carrying out a task in practice, detailed enough that someone could follow them directly.

- **Guideline** — recommended, but not mandatory, best practices — helpful suggestions rather than hard requirements.

Memory trick: Policy = what and why, Standard = specific mandatory requirement, Procedure = exact steps, Guideline = optional suggestion.

Risk Management

Risk management is the structured process of understanding what could go wrong, how likely and how damaging it would be, and deciding what to actually do about each identified risk — this is the discipline that ultimately decides where a limited security budget gets spent.

- **Risk identification** — the process of actively finding what could go wrong in the first place, essentially combining known threats with known vulnerabilities.
- **Risk assessment** — evaluating each identified risk's likelihood of occurring and its potential impact if it does, in order to understand which risks actually matter most.
- **Qualitative assessment** — rating risks using descriptive labels (low/medium/high) — faster to perform and easier for non-technical stakeholders to understand, but more subjective.
- **Quantitative assessment** — rating risks using actual numbers and dollar values — more precise and easier to justify budget decisions with, but requires significantly more data to calculate accurately.
- **Risk register** — a tracked, living document that lists every identified risk along with its rating, its owner, and its current treatment status — this is the practical tool that keeps risk management from being just a one-time exercise.

Once a risk is identified and assessed, an organization has exactly four ways to actually respond to it:

- **Avoid** — stop doing the risky activity altogether, eliminating the risk by simply not engaging in whatever creates it.
- **Transfer** — shift the financial consequence of the risk elsewhere, most commonly by purchasing cybersecurity insurance.
- **Mitigate** — reduce the risk's likelihood or potential impact through security controls, without fully eliminating the underlying activity.
- **Accept** — formally acknowledge the risk and choose to take no further action, usually because the cost of additional controls would outweigh the risk itself.

Compliance & Regulations

Different laws and standards apply depending on the type of data an organization handles and where it operates — the exam tests broad recognition of these, not deep legal expertise.

- **GDPR** — a European Union regulation governing how the personal data of EU citizens must be collected, handled, and protected, with significant penalties for violations even by companies based outside the EU.
- **HIPAA** — a United States regulation specifically protecting health information (PHI), applying to healthcare providers, insurers, and their business associates.
- **PCI-DSS** — an industry standard (not a government law) created by the payment card industry itself, applying to any organization that stores, processes, or transmits credit card data.

These are broad strokes only — compliance requirements are detailed and jurisdiction-specific in practice; treat this section as 'know that each one exists and roughly what category of data it protects,' not as a legal reference to rely on.

Audits & Assessments

Audits exist to independently verify that an organization's actual practices match what its policies and required standards claim they should be.

- Internal audits are performed by the organization's own team, typically for ongoing self-improvement.
- External audits are performed by an independent third party, providing an outside, less biased perspective.

- Regulatory audits specifically check compliance against a particular law or required standard (like PCI-DSS).
- Audit findings typically result in a formal remediation plan with concrete deadlines, rather than just being noted and left unaddressed.

Third-Party / Vendor Risk Management

Any external vendor with access to your systems or data effectively extends your own attack surface — their security weaknesses become your risk too, which directly connects back to the supply chain vulnerabilities discussed in Domain 2.

- Vendor risk assessments evaluate a potential vendor's security posture before granting them access, rather than simply trusting them by default.
- Contractual security requirements and right-to-audit clauses give an organization formal, enforceable leverage to hold vendors accountable to agreed security standards over time.

Security Awareness Training

Effective security awareness training is ongoing, not a one-time onboarding checkbox — regular training, including realistic phishing simulations, keeps security genuinely top-of-mind for everyday employees, who consistently remain the most targeted part of any organization's overall attack surface, precisely because human judgment can be manipulated in ways a firewall never can.

EXAM TIP

Policy vs Standard vs Procedure vs Guideline gets tested directly — know the hierarchy and practice sorting a given real-world example into the right bucket.

The four risk responses — Avoid, Transfer, Mitigate, Accept — are a very common multiple-choice pattern. Practice spotting which one a described scenario actually represents.

You don't need deep legal knowledge of GDPR/HIPAA/PCI-DSS for the exam — just know what kind of data each one protects and roughly who it applies to.

How to Use These Notes

- Read one domain at a time — don't try to absorb all five in one sitting; the concepts build on each other and reward slower, deeper reading.
- After each domain, close the notes and try to explain each heading out loud in your own words, as if teaching it to someone else. If you can't, that's your gap — go back and reread just that section.
- Pair this with hands-on practice — a home lab, TryHackMe's Security+ path, or timed practice exams — since Security+ increasingly tests applied scenarios, not just bare definitions.
- Revisit the exam-tip boxes right before your test date as a fast final review, once the fuller explanations have already sunk in.

"Complex ko simple. Simple ko powerful." — MR CYBER